

Midterm Evaluation Report: Design of High-Performance and Area-Efficient Decoder for 5G LDPC Codes

Course: VLS822 - Physical Design of ASICs

Instructor: Dr. Anuj Verma

Team Members:

Sahil Singh, Shrujan Teja, Rajdeep Saha,
Suhas Nagabandi, Nachiappan N

March 14, 2026

1 Introduction and Paper Specification

The 5G New Radio (NR) standard employs Low-Density Parity-Check (LDPC) codes for error correction in the data channel. The assigned paper, "*Design of High-Performance and Area-Efficient Decoder for 5G LDPC Codes*," addresses a vulnerability in the 5G base graph: the high proportion of fragile "degree-1" variable nodes. If a check node passes an inaccurate Check-to-Variable (CTV) message to a degree-1 node, it can corrupt the entire decoding process.

To solve this, the paper proposes the **Improved Adapted Min-Sum (IAMS)** algorithm, which utilizes both the first minimum (min_1) and second minimum (min_2) incoming messages to calculate highly accurate CTV outputs.

For this midterm evaluation, our team focused strictly on establishing the complete, unoptimized **baseline datapath** of the decoder utilizing the IAMS algorithm, ensuring functional correctness and standard cell synthesizability before introducing area optimizations.

2 Detailed Hardware Architecture

2.1 Midterm Deliverable: Simplified Baseline Architecture

To adhere to the midterm requirement of an unoptimized implementation, complex architectural features such as the split-storage memory and selective-shift network were bypassed for this evaluation. While these optimized modules have already been drafted in our Verilog code, our testbench and synthesis scripts explicitly isolate and instantiate

only the standard, unoptimized baseline datapath (`ldpc_decoder_layer`). The implemented baseline hardware architecture follows a pure, sequential 5-stage loop.

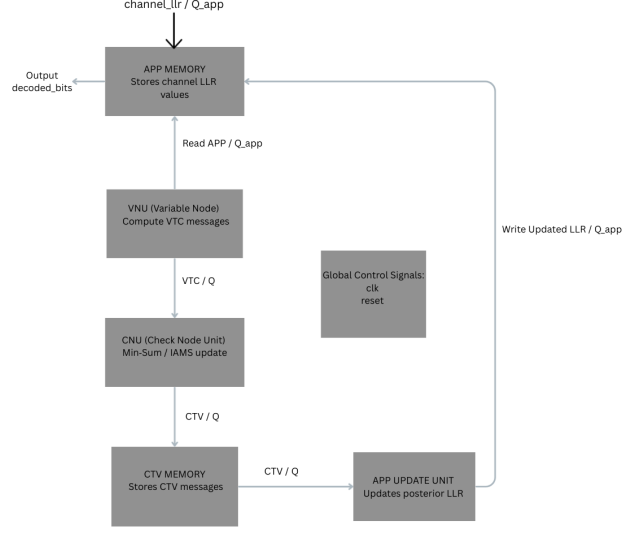


Figure 1: Implemented Baseline LDPC Decoder Datapath (Unoptimized)

2.2 Endterm Roadmap: Optimized Architecture

The goal for the endterm is to integrate the paper’s area optimizations, including the selective-shift routing networks and the split-storage CTV memories.

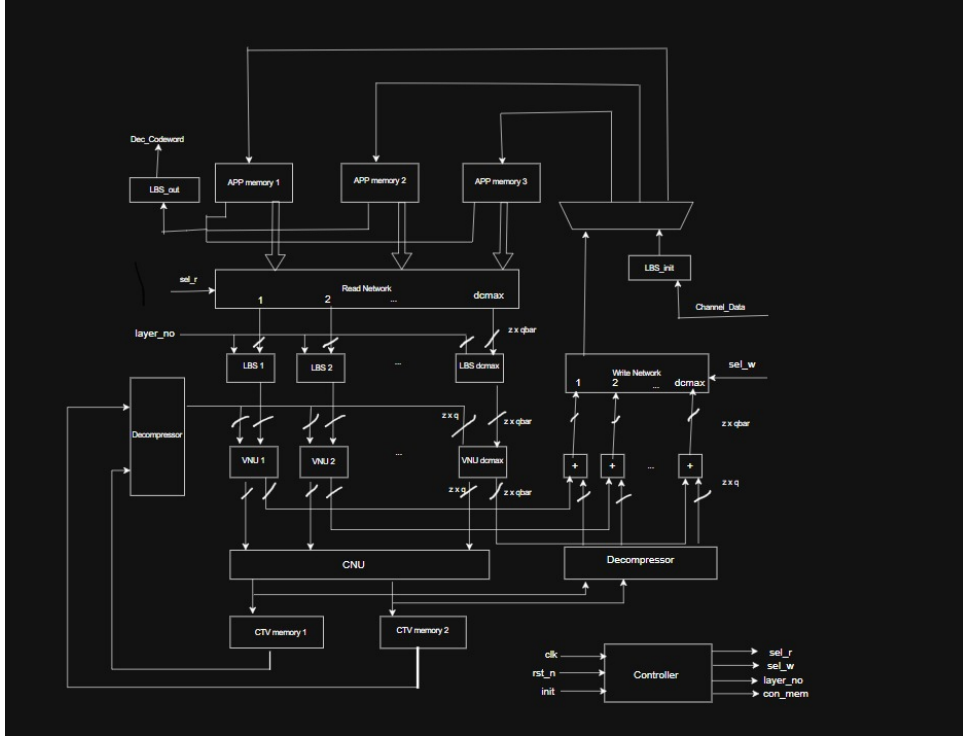


Figure 2: Target Fully Optimized Architecture with Split Memory and Selective-Shift

3 RTL Implementation and Code Flow

The design was modeled following a modular approach.

- **Top Module (ldpc_decoder_layer):** Acts as the master wrapper, instantiating the memory arrays and routing buses between the processing units.
- **Sub-modules:**
 - **vnu_update:** Implements the arithmetic subtraction and bit-width clamping.
 - **cnu_update_iams:** Contains the compare-and-select logic tree to isolate min_1 and min_2 .
 - **app_memory & ctv_memory:** Standard register arrays parameterized by lifting size and quantization bits.
- **Testbench (LDPC_test.vh):** To enforce the baseline constraints, it directly instantiates the **ldpc_decoder_layer** and acts as a sequential state machine, driving the clock, reset, and channel LLR signals to verify 7 distinct test vectors.

4 Functional Verification & Waveform Analysis

Simulation was executed using Icarus Verilog (**iverilog**) and visualized via GTKWave. The testbench successfully passed all embedded functional checks.

```
VCD info: dumpfile LDPC_tb.vcd opened for output.
--- TEST 1: cn_update_iams ---
1a) n=idx1 expect|ctv|=2 got ctv=2 PASS
1b) OMS core+highDeg expect|ctv|=0 got ctv=0 PASS
1c) n=idx2 expect|ctv|=1 got ctv=1 PASS
1d) min1==min2 other expect|ctv|=2 got ctv=2 PASS
TEST 1 done.

--- TEST 2: vn_update ---
2a) t=1 beta_tilde=10 expect 10 PASS
2b) beta=7 vtc=7 expect 7,7 PASS
2c) clamp high vtc=7 expect 7 PASS
2d) clamp low vtc=-8 expect -8 PASS
TEST 2 done.

--- TEST 3: app_update_unit ---
3a) 3+5=8 expect 8 PASS
3b) pos sat=31 expect 31 PASS
3c) neg sat=-32 expect -32 PASS
3d) zero=0 expect 0 PASS
TEST 3 done.

--- TEST 4: ctv_memory ---
4a) wrote -3 read=-3 expect -3 PASS
4b) wrote +7 read=7 expect +7 PASS
4c) retain read=-3 expect -3 PASS
TEST 4 done.

--- TEST 5: split_storage_unit ---
5a) min1=2 min2=5 idx1=7 signs=1010101010 PASS
5b) overwrite min1=0 min2=1 expect 0,1 PASS
TEST 5 done.

--- TEST 6: selective_shift_network ---
6a) nz=0 out=0x0000 expect 0x0000 PASS
6b) sh=0 out=0xabcd expect 0xabcd PASS
6c) sh=1 in=0x1234 out=0x2341 (circular left shift Q bits)
TEST 6 done.

--- TEST 7: ldpc_decoder_layer ---
7a) Waiting for done (strong +LLR) ...
7a) DONE after 632 cycles.
7b) decoded_bits=0xxxxxxxxxxxxx INFO(non-zero without full H-matrix)
7c) Waiting for done (strong -LLR) ...
7c) DONE after 632 cycles. decoded_bits=0x0000000000Xxx
TEST 7 done.

=====
Testbench complete. FAIL count = 0
=====

LDPC_test.vh:542: $finish called at 12785000 (1ps)
sahil_singh@zaptor:~/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ gtwave LDPC_tb.vcd
```

Figure 3: Icarus Verilog Compilation and Testbench Execution (All Tests Passed)

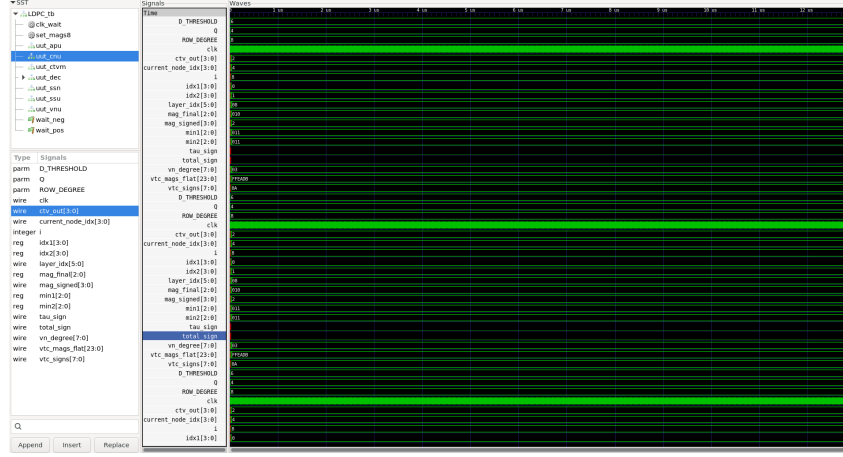


Figure 4: Module-wise Simulation: Check Node Unit (CNU) calculating min_1 and min_2

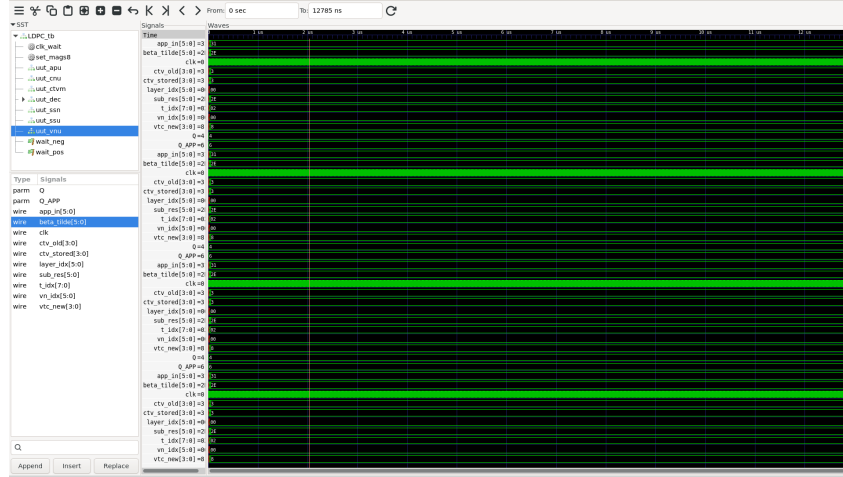


Figure 5: Module-wise Simulation: Variable Node Unit (VNU) subtraction and clamping

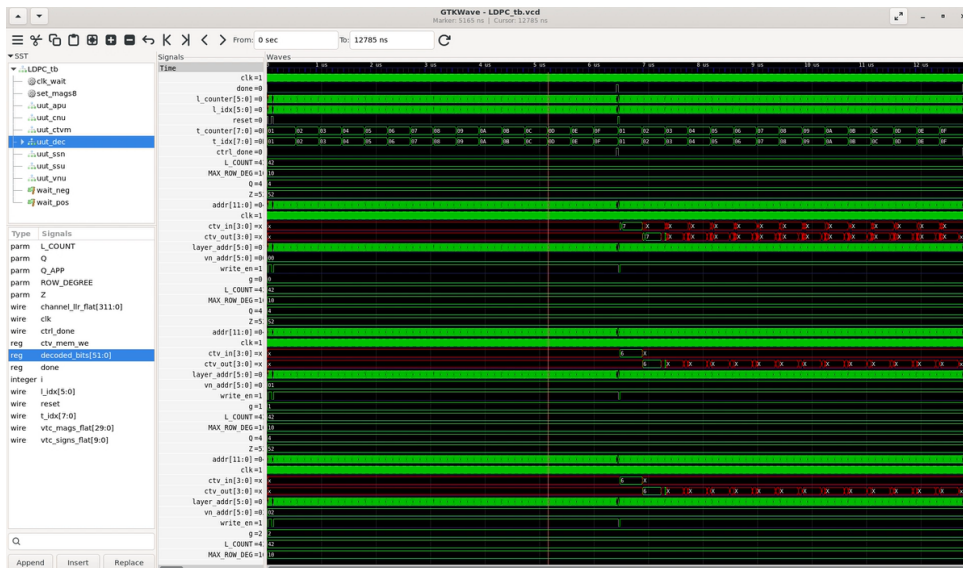


Figure 6: An instance of the overall System Waveform: Top-Level Decoder

5 Logic Synthesis and Implementation

To prove hardware viability, the verified baseline RTL was synthesized using the open-source **Yosys** synthesis suite, targeting the **SkyWater 130nm** standard cell library (sky130_fd_sc_hd_tt_025C_1v80.lib).

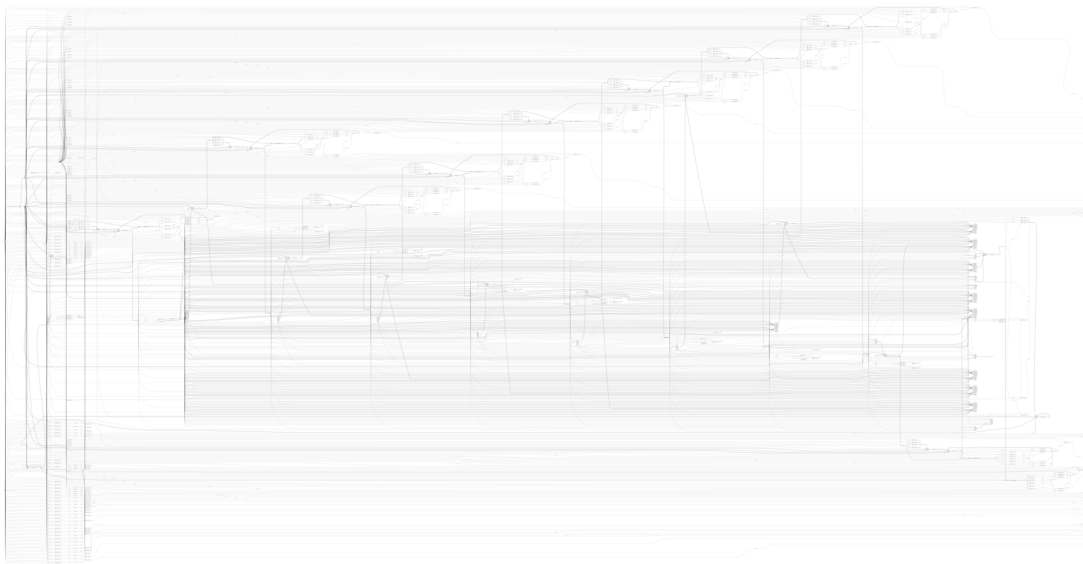


Figure 7: Synthesized Gate-Level Schematic of the Baseline Datapath

```
=== ldpc_decoder_layer ===
+-----Local Count, excluding submodules.
|
165 wires
1107 wire bits
123 public wires
1015 public wire bits
5 ports
367 port bits
308 cells
  1  $_ANDNOT_
 40  $_DFFE_PN_
 94  $_DFFE_PP_
 61  $_DFF_P_
 80  $_MUX_
  1  $_NOT_
 10  $_OR_
  1  $_SDFFE_PP0P_
 20  $_XOR_
41 submodules
10  $paramod$6608040f2584795fab68e1ec2b8fc9e70dd56d7b\ctv_memory
10  $paramod$c2a956b28df0e100e8b4efc939e5d4ebbd35ca37\app_update_unit
10  $paramod$c2a956b28df0e100e8b4efc939e5d4ebbd35ca37\vn_update
  1  $paramod$e2791f96de9edc6df9b551b34e88dc22c6632c6f\ldpc_merged_controller
 10  $paramod$fe5d0498c8b9a1ae9375c2e561534f0dd3bd6766\cn_update_iam
```

Figure 8: Yosys Terminal Output detailing Sky130 Standard Cell Utilization

```
ldpc_baseline_netlist.v X
5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > 369D5C8A42C1FF9CB775B3162C5222D3F80D0794 > transfers > 2026-11 >
152436 module ldpc_decoder_layer(clk, reset, channel_llr_flat, decoded_bits, done);
152437     input clk;
152438     wire clk;
152439     input reset;
152440     wire reset;
152441     input [311:0] channel_llr_flat;
152442     wire [311:0] channel_llr_flat;
152443     output [51:0] decoded_bits;
152444     wire [51:0] decoded_bits;
152445     output done;
152446     wire done;
152447     wire [5:0] _000_;
152448     wire [5:0] _001_;
152449     wire [5:0] _002_;
152450     wire [5:0] _003_;
152451     wire [5:0] _004_;
152452     wire [5:0] _005_;
152453     wire [5:0] _006_;
152454     wire [5:0] _007_;
152455     wire [5:0] _008_;
152456     wire [5:0] _009_;
152457     wire _010_;
152458     wire _011_;
152459     wire _012_;
```

Figure 9: Generated Physical Netlist showing top level module

6 Endterm Roadmap: Area Optimization & Physical Design

With the unoptimized baseline fully verified and synthesized, the endterm phase will focus exclusively on implementing the paper’s proposed area optimizations and pushing the design through the RTL-to-GDSII flow using OpenROAD/OpenLane.

1. **Layer Merging:** Re-introducing the complex top controller to schedule and process two orthogonal layers of the base graph simultaneously.
2. **Split Storage Method:** Replacing the standard CTV memory array with the custom dual-structure (CTV Memory 1 and 2), targeting a reduction in memory area.
3. **Selective-Shift Structure:** Eliminating massive crossbar routing networks by utilizing optimized cyclic shift registers for APP message passing.